De mirar imágenes a leer secuencias

Neural Networks: Deep Learning · Arquitecturas



UIDE · Maestría en Ciencia de Datos

Semana 2

Andrés Ordóñez-Bolaños, MSc

Viernes 6 · 18:00–20:00 · Sábado 7 · 08:00–12:00

Noviembre de 2026 · Microsoft Teams

La semana en dos sesiones

Dos preguntas: **¿cómo mira una red una imagen?** y **¿cómo lee una secuencia?**
Más corta que la semana 1 y con más tiempo de práctica.

Viernes 6 18:00–20:00 · imágenes

- 18:00 **Pre-quiz**
- 18:09 **00 · Keras (pendiente)**
- 18:22 **01 · Cómo mira una imagen**
- 18:54 **Práctica NB03**
- 19:29 **Densa contra CNN**
- 19:40 **Cierre y lectura**

Sábado 7 08:00–12:00 · secuencias y atención

- 08:00 **Repaso**
- 08:08 **02 · Secuencias**
- 08:43 **Práctica NB04**
- 09:30 Pausa
- 09:45 **03 · Atención**
- 10:25 **04 · Elegir y tu proyecto**
- 11:00 **Quiz final**

práctica en Colab · quiz tipo Kahoot · lectura de tendencia

Antes de empezar: ¿de dónde partimos?

8 preguntas cortas tipo Kahoot. **No es nota**: es tu punto de partida para medir cuánto avanzas esta semana.

1. Escanea el código o abre **deeplearningweek1.vercel.app/?pre2**
2. Escribe tu nombre (el mismo de la semana 1) y responde rápido: **más rapidez, más puntos**
3. Al final, copia tu código y **pégalo en el chat de Teams**

✓ Si no sabes una, elige la que te parezca. Equivocarse aquí es parte del plan.

deeplearningweek1.vercel.app/?pre2

El sábado vas a poder explicar estas 5 ideas

Las medimos dos veces: en el pre-quiz de hoy y en el quiz final del sábado.

1. Cada arquitectura es una suposición sobre los datos
2. Qué hace un filtro: una lupa que recorre la imagen
3. Por qué una CNN necesita muchos menos parámetros
4. Qué guarda una RNN y por qué se olvida
5. Qué hace la atención y cuál es su precio

Hoy: **1, 2 y 3**. Mañana: **4 y 5**. Cada una tiene su lámina con la etiqueta amarilla **Idea clave**.

00

Lo que quedó pendiente: Keras

La semana pasada hicimos la red a mano. Keras la arma, la entrena y la usa en diez líneas.

- Motor y volante
- Los 4 pasos
- Mírala aprender
- El NB02 en casa

Viernes · 13 minutos

TensorFlow y Keras: el motor y el volante

Dos librerías de Python que ya vienen en Colab. Con Keras manejas; TensorFlow hace las cuentas por debajo, incluida la retropropagación.

Tú · Python escribes qué quieres en Colab

↓

Keras **el volante**: capas, pérdidas y `fit`

↓

TensorFlow **el motor**: tensores y culpas calculadas solas

↓

La máquina la CPU o GPU hace las multiplicaciones

```
import tensorflow as tf   # the engine
import keras              # the wheel
from keras import layers  # the pieces
```

Puedes manejar sin abrir el capó

Keras arma y entrena una red en diez líneas. Saber que existe el motor ayuda cuando algo falla: por eso la semana pasada hicimos las cuentas a mano.

`2015` nacen Keras (François Chollet, marzo) y TensorFlow (Google, noviembre)

Keras en 4 pasos: construir, compilar, entrenar y predecir

El problema de las lunas: 600 puntos de dos clases, cada uno con 2 datos. La misma receta sirve para fotos (hoy) y para secuencias (mañana).

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")])
model.compile(optimizer=keras.optimizers.Adam(0.01),
              loss="binary_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=60,
                    batch_size=32, validation_split=0.2)
p = model.predict(NUEVOS)
```

1 · Construir — apilar capas como LEGO: ReLU dentro, sigmoide a la salida

2 · Compilar — Adam baja la montaña; la BCE mide qué tan mal vamos

3 · Entrenar — `fit`: adelante, culpa hacia atrás y un paso, 60 vueltas

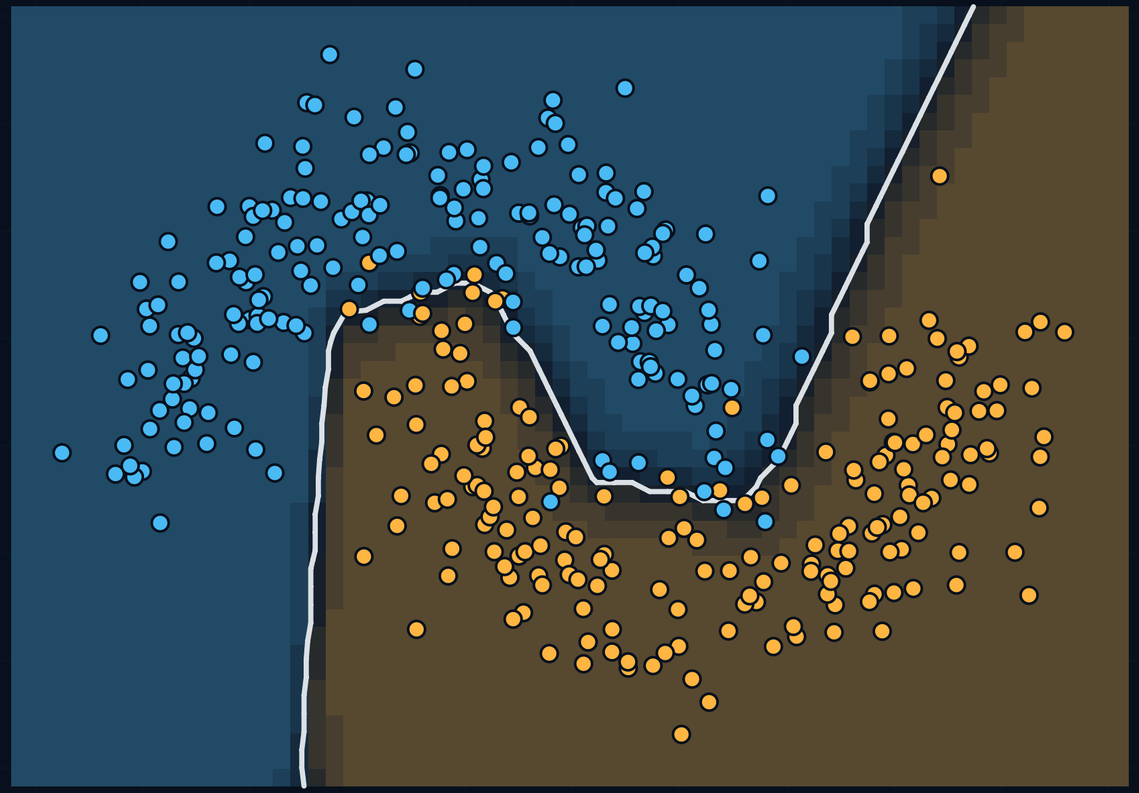
4 · Predecir — una probabilidad; con el umbral 0.5, sí o no

Diez líneas, **337** parámetros y **95 %** en el examen final.

Mira a la red aprender: `fit` en vivo

```
history = model.fit(X_train, y_train, epochs=60, batch_size=32, validation_split=0.2)
```

`epochs=60` 60 vueltas · `batch_size=32` lotes de 32 · `validation_split=0.2` el simulacro que la red no estudia



época **60**/60 paso **720**/720 pérdida **0.09**
simulacro **96** %

Entrenar 60 épocas | **Reiniciar**



El NB02 queda para casa

En casa · sin nota

30 minutos

Si el sábado no alcanzaste, empieza por las partes **E** y **F**: son las de esta lámina y la anterior.

Qué deberías ver

- **E** · Keras paso a paso: **XOR resuelto** con 33 parámetros
- **F** · Las lunas: **337** parámetros y **95 %** en el examen final
- **Experimento 2** · los círculos con más de **90 %**

Puente: las lunas eran una tabla de **2 columnas**. ¿Y si la entrada es una foto de **784 números**?

deeplearningweek1.vercel.app/?nb02

Archivo → **Guardar una copia en Drive**
y ejecuta primero **Configuración**

01

Cómo mira una red una imagen

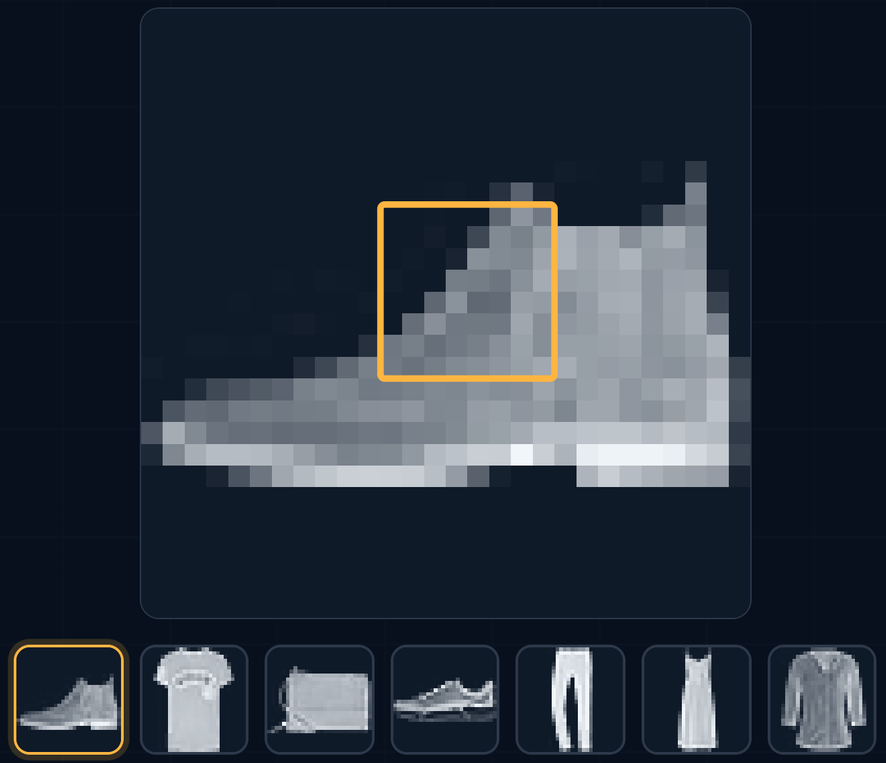
Una foto pequeña ya tiene 784 números. ¿Conectamos todos con todos, o miramos de cerca?

- Una foto son números
- La densa aplana
- La suposición
- La lupa
- Mapas
- Compartir
- Pooling
- En Keras
- Softmax

Viernes · 32 minutos

Una imagen es una tabla de números

Fashion-MNIST (Zalando, 2017): 70 000 fotos de ropa de 28 × 28 píxeles en 10 clases. Cada píxel es un brillo: **0 es negro y 255 es blanco**.



botín · filas 9–16, columnas 11–18

0	0	1	0	0	88	143	110
0	0	4	0	53	129	120	147
0	2	0	11	137	130	128	160
3	0	0	115	114	106	137	168
0	0	89	139	90	94	153	149
0	98	136	110	109	110	162	135
108	117	99	111	117	136	156	134
111	103	115	129	134	143	154	165

Pasa el mouse por la foto para mover el recuadro.

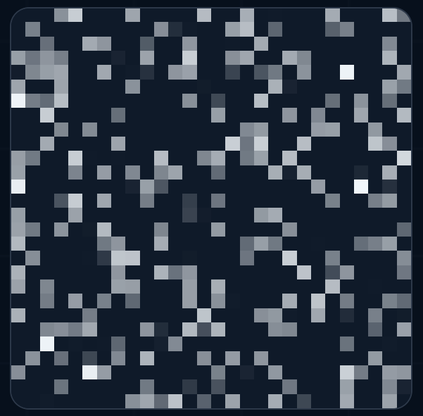
28 × 28 = **784** números por foto. La foto **es** esa tabla: no hay nada más.

La red densa aplana la foto y pierde la vecindad

Una `Dense` conecta **cada** píxel con **cada** neurona. Funciona, pero no sabe qué píxel está al lado de cuál.



La foto · 28 × 28



Los mismos píxeles, mezclados

Para una capa `Dense`, la foto llega como **una fila de 784 números**: la fila 1, después la 2, y así hasta la 28.

Si mezclamos los píxeles (siempre igual en todas las fotos), la densa aprende **exactamente lo mismo**: `87.2 %` contra `86.7 %`. Nunca supo qué píxel era vecino de cuál.

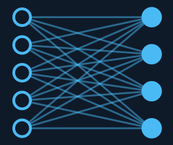
Lo que recibe la red densa · 784 números en fila (cada rayita es un píxel)



Cada arquitectura es una suposición sobre los datos

Idea clave 1

Una arquitectura no es solo una lista de capas: es una apuesta sobre **cómo están organizados** los datos.



Densa

Supone que **todo se relaciona con todo**.

Tablas: los socios de una cooperativa, cada columna un dato.



Convolucional · CNN

Supone que **lo cercano importa** y que el mismo patrón puede estar **en cualquier parte**.

Imágenes: una lupa que recorre la foto.



Recurrente · RNN

Supone que **el orden importa** y resume lo leído en una libreta.

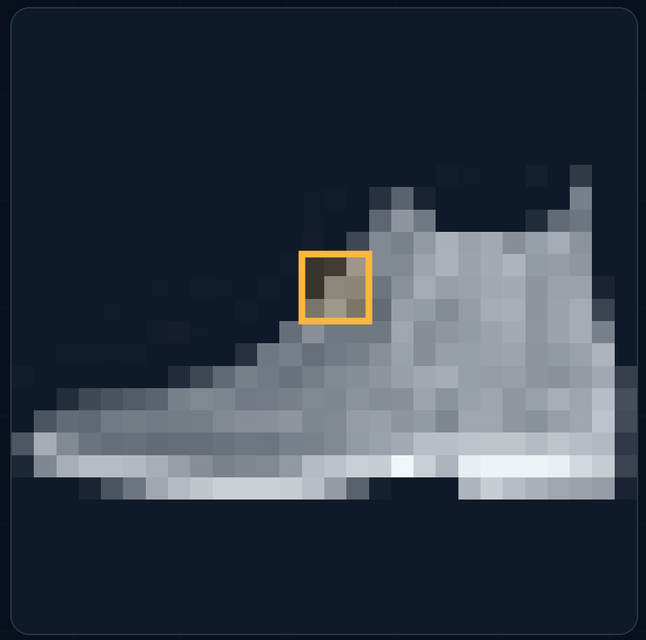
Secuencias: frases, pagos de cada mes. Mañana.

Si la suposición es cierta, la red **aprende más con menos**. Si no lo es, le estorba. Elegir arquitectura = elegir qué suponemos.

La lupa: un filtro de 3 × 3 recorre la imagen

Idea clave 2

En cada lugar multiplica **9 píxeles por 9 pesos** y suma. Donde encuentra su patrón, el resultado es grande. Eso es una **convolución**.



La foto · el recuadro es la lupa de 3 × 3

9 píxeles

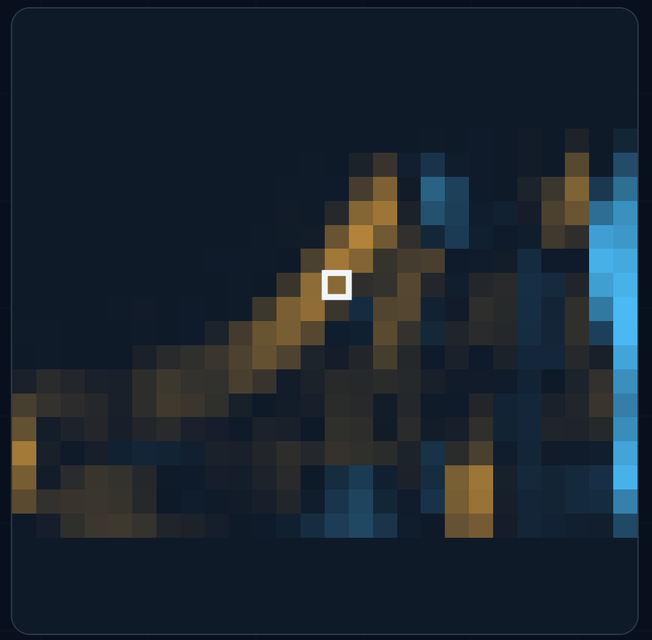
0.00	0.04	0.54
0.00	0.45	0.45
0.35	0.55	0.35

×

-1	0	+1
-1	0	+1
-1	0	+1

Borde vertical
oscuro a la izquierda, claro a la derecha

= suma
+0.99



El mapa · 26 × 26: dónde encontró el patrón

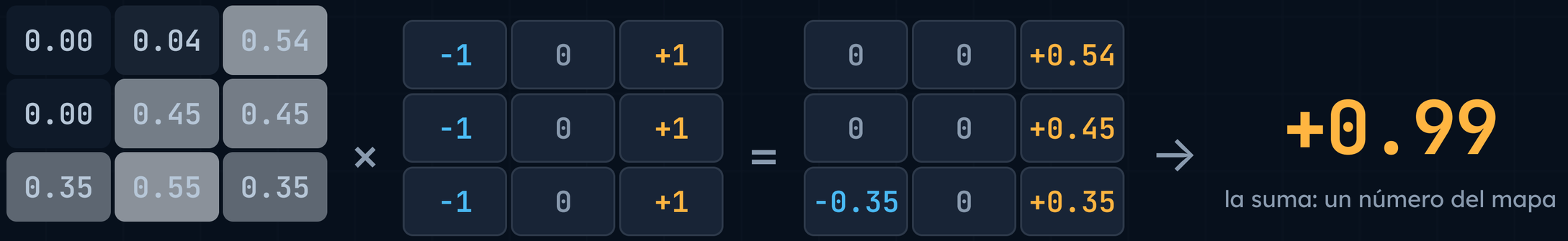
Borde vertical | Borde horizontal | Desenfoque

▶ Recorrer | Un paso | Todo

Un filtro, con números

Idea clave 2

La misma cuenta de la neurona de la semana 1: cada dato por su peso, y todo sumado. Solo que ahora los datos son **9 píxeles vecinos**.



9 píxeles del botín (filas 11–13, columnas 13–15)

filtro de borde vertical

casilla por casilla

$$\underbrace{z}_{\text{un número del mapa}} = \underbrace{\sum_{9} \text{píxel} \cdot \text{peso}}_{\text{9 productos sumados}} + \underbrace{b}_{\text{sesgo}}$$

Léelo así: la columna de la izquierda (oscura: 0, 0 y 0.35) resta; la de la derecha (clara: 0.54, 0.45 y 0.35) suma. Queda **+0.99**: ahí hay un borde «oscuro → claro».

Muchos filtros, muchos mapas: lo que aprendió la red

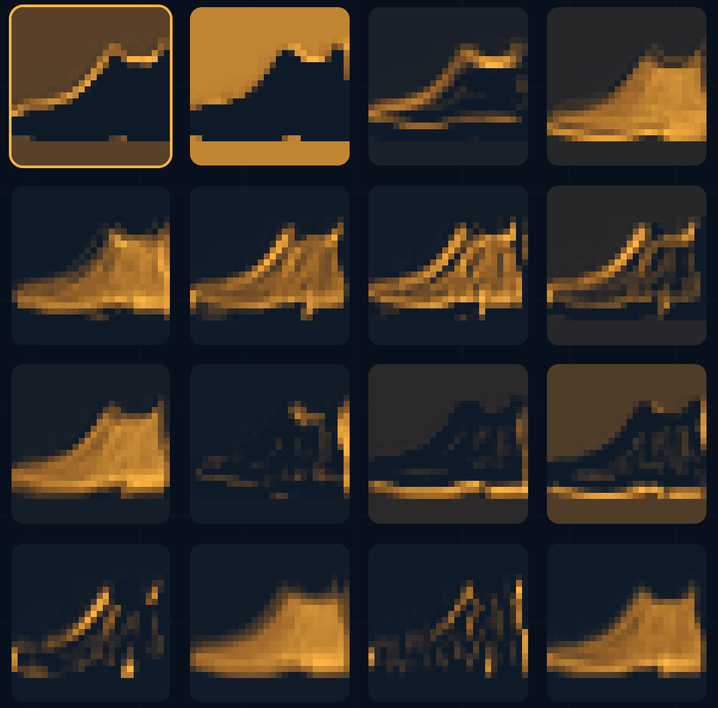
La primera capa de la CNN del NB03 tiene **16 filtros**. Nadie se los dio: los aprendió de 54 000 fotos. Cada mapa dice **dónde** encontró su filtro el patrón.

16 filtros aprendidos · 3 × 3

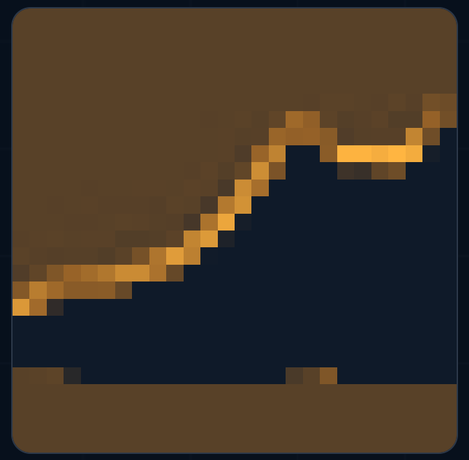


ámbar = peso positivo · azul = negativo

Sus 16 mapas sobre el botín



-0.45	-0.20	+0.06
-0.29	-0.17	+0.12
+0.15	+0.19	-0.03



Filtro 1. Parece un **detector de bordes horizontales**: la suela, la caña. Nadie lo programó: salió del entrenamiento.

El mismo filtro en toda la imagen Idea clave 3

Un filtro de 3 × 3 tiene **9 pesos y 1 sesgo**, y sirve para las 676 posiciones de la foto.
Es como un sello que se usa en toda la hoja.

Primera capa densa · 128 neuronas

100 480

784 × 128 + 128

Cada neurona tiene un peso para **cada** píxel.

Primera capa convolucional · 16 filtros

160

16 × (3 × 3 + 1)

Cada filtro tiene 10 números y **recorre toda la foto**.

Red completa: la densa aprende **101 770** números; la CNN, **12 810**. **Unas 8 veces menos**, porque comparte sus filtros.

Además, si el borde se mueve de lugar, **el mismo filtro lo encuentra**. Eso lo vamos a probar en el NB03.

Pooling: quedarse con lo más fuerte

`MaxPooling2D(2)` : de cada ventana de 2 × 2 se queda con el **número mayor**. El mapa queda 4 veces más chico y tolera que la prenda se mueva un poco.



Con un mapa real · el borde vertical del botín



26 × 26 = 676 números

13 × 13 = 169

La CNN en Keras: los mismos 4 pasos, capas nuevas

Cada `Conv2D` es un juego de filtros; cada `MaxPooling2D`, un resumen. Al final, la densa de siempre decide entre 10 prendas.

```
model = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),  # gray photo
    layers.Conv2D(16, 3, activation="relu"),
    layers.MaxPooling2D(2),          # max of 2x2
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(2),
    layers.Flatten(),                # 800 numbers
    layers.Dense(10, activation="softmax"),
])
```



capa	forma de salida	parámetros
Input	28 × 28 × 1	–
Conv2D(16)	26 × 26 × 16	160
MaxPool	13 × 13 × 16	0
Conv2D(32)	11 × 11 × 32	4 640
MaxPool	5 × 5 × 32	0
Flatten	800	0
Dense(10)	10	8 010

Total: 12 810 parámetros · la densa: 101 770

De sí o no a 10 clases: softmax

La sigmoide de la semana 1 daba una probabilidad (sí o no). **Softmax** da 10, una por prenda, y **suman 1**. La red elige la más alta.



La respuesta correcta: **camisa**



CNN · 12 810 parámetros

Densa · 101 770

camiseta	11 %
pantalón	0 %
suéter	5 %
vestido	1 %
abrigo	5 %
sandalia	0 %
camisa	77 %
zapatilla	0 %
bolso	0 %
botín	0 %

Las 10 probabilidades suman `1.00`. La red elige la más alta.

$$\underbrace{p_k}_{\text{probabilidad de la prenda k}} = \frac{e^{z_k}}{\underbrace{\sum_j e^{z_j}}_{\text{su puntaje entre todos}}}$$

Léelo así: cada puntaje se vuelve positivo con e^z y se divide entre la suma de todos: quedan 10 porciones de una torta.

La camisa: la CNN duda entre **camisa 77 %** y camiseta 11 %. Duda entre prendas que se parecen, como una persona.

En `compile`: `loss="sparse_categorical_crossentropy"`, la BCE de la semana 1 para muchas clases.

Práctica en clase: NB03

En clase · sin nota

35 minutos

En parejas. Todo el código ya viene escrito y comentado. Antes de cada celda, apuesta qué va a salir.

Qué deberías ver

- ◯ **0** · La foto es una tabla: `(60000, 28, 28)`, de 0 a 255
- ◯ **1** · Tu lupa a mano: la suma da **+0.99**
- ◯ **2** · La densa: **101 770** parámetros y **86.7 %**
- ◯ **3** · La CNN: **12 810** parámetros y **87.7 %**
- ◯ **5** · Mover 2 píxeles: densa **56.5 %**, CNN **75.2 %**
- ◯ **6** · Lo que más confunde: **camisa ↔ camiseta**

deeplearningweek1.vercel.app/?nb03

Archivo → **Guardar una copia en Drive**
y ejecuta primero **Configuración**

Densa contra CNN: lo que vimos en el NB03

Mismas fotos, mismos 5 épocas, mismo optimizador. Solo cambia la suposición.

1 · **Parámetros** que aprende

Densa **101 770**

CNN **12 810**

8 veces menos: la CNN comparte sus filtros.

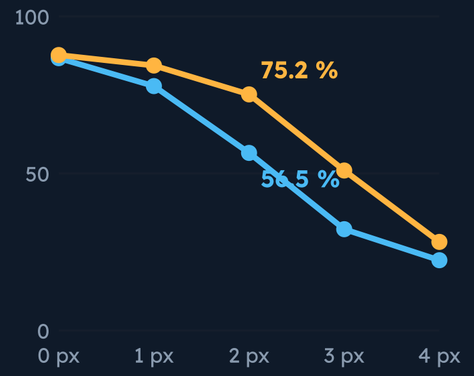
2 · **Examen final** · 10 000 fotos

Densa **86.7 %**

CNN **87.7 %**

La CNN acierta un poco más con mucho menos.

3 · **Mover la prenda** unos píxeles



Con 2 px la densa se cae; la CNN aguanta. Con 4 px caen las dos.

4 · **Mezclar los píxeles** (siempre igual)

Densa **87.2 %**

CNN **85.2 %**

A la densa le da igual. A la CNN le duele: su suposición ya no es cierta.

La CNN gana porque **su suposición es cierta** para las fotos. Con 4 píxeles las dos se caen: no es magia. La semana 3 trata de **robustez**.

De los cheques a ImageNet

Las CNN tienen más de 40 años. Dominaron la visión desde 2012 y siguen vivas.

1980
Fukushima · el neocognitrón: capas que detectan rasgos

1989
LeCun · una CNN lee códigos postales escritos a mano

1998
LeNet-5 · lee cheques en los bancos

2012
AlexNet · gana ImageNet con GPU: el error cae de 26 % a 15 %

2015
ResNet · 152 capas, 3.6 % de error

2020
Vision Transformer · la atención llega a las imágenes

Lectura de tendencia

En 2023, Google DeepMind entrenó CNN con **el mismo presupuesto de cómputo** que los Vision Transformers más fuertes: empate técnico, más de 90 % en ImageNet. «No hay evidencia fuerte de que los ViT superen a las CNN cuando se evalúan con justicia» (Smith et al., 2023).

Lo que te llevas hoy

La suposición	**cada arquitectura apuesta por cómo están organizados los datos**
La lupa	**un filtro de 3 × 3 recorre la foto y responde donde está su patrón**
Compartir	**el mismo filtro en toda la foto: 8 veces menos parámetros**

Para mañana

Lee la lectura de tendencia **«¿El regreso de las redes recurrentes? Transformers, Mamba y la búsqueda de la arquitectura ideal»** (5 páginas) y responde la encuesta en Canvas.

Mañana: si los datos llegan **en orden**, como las palabras de una frase, ¿cómo los lee una red?

Redes que leen en orden

Secuencias, memoria y atención

UIDE · Maestría en Ciencia de Datos | Semana 2 · Sábado

Andrés Ordóñez-Bolaños, MSc

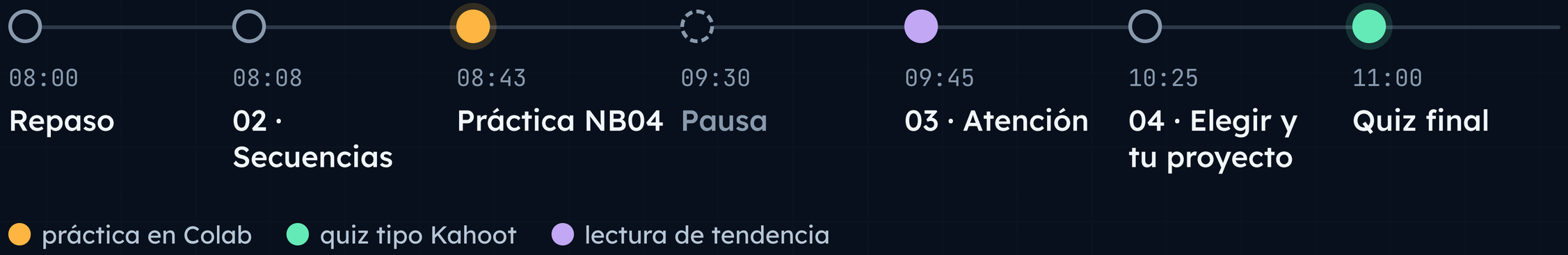
Sábado 7 de noviembre · 08:00–12:00

Microsoft Teams



El sábado en cuatro paradas

Secuencias y la libreta, práctica, atención y la tendencia, y elegir la arquitectura de tu proyecto.



El viernes en tres ideas

¿Quién explica cada una con sus palabras, antes de que aparezca?

1 · La suposición	**la densa supone que todo se relaciona con todo; la CNN, que lo cercano importa**
2 · La lupa	**un filtro de 3 × 3 recorre la foto: 9 píxeles por 9 pesos, sumados**
3 · Compartir	**el mismo filtro en toda la foto: 12 810 parámetros contra 101 770**

La prueba: al mover la prenda 2 píxeles, la densa cayó a `56.5 %` y la CNN aguantó `75.2 %`. Hoy la pregunta cambia: **¿y si importa el orden?**

02

Cómo lee una red una secuencia

En una frase, en un historial de pagos o en una canción, el orden cambia el significado.

Datos en orden | El orden importa | La libreta | La cuenta | Desenrollar | El olvido | LSTM | En Keras | Práctica NB04

Sábado · 35 minutos y 40 de práctica

Datos que llegan en orden

Una **secuencia** es una tabla en la que el orden de las filas importa. Su forma: **(pasos, datos por paso)**.

Texto

`el` `socio` `pagó` `la` `cuota`

`(5, 1)`

palabra tras palabra

Historial de pagos

`3` `5` `3` `12` `7` `…` `24`

`(12, 1)`

días de atraso de cada mes

Ventas de una agencia

`lun` `mar` `mié` `jue` `vie`

`(365, 1)`

un número por día

Voz

`0.01` `-0.2` `0.4` `…`

`(16 000, 1)`

16 000 números por segundo de audio

Sensores

`temp` `presión` `…`

`(1 440, 2)`

cada minuto, 2 datos: pasos × datos por paso

Video

`foto 1` `foto 2` `foto 3` `…`

`(30, 28, 28)`

30 fotos por segundo, en orden

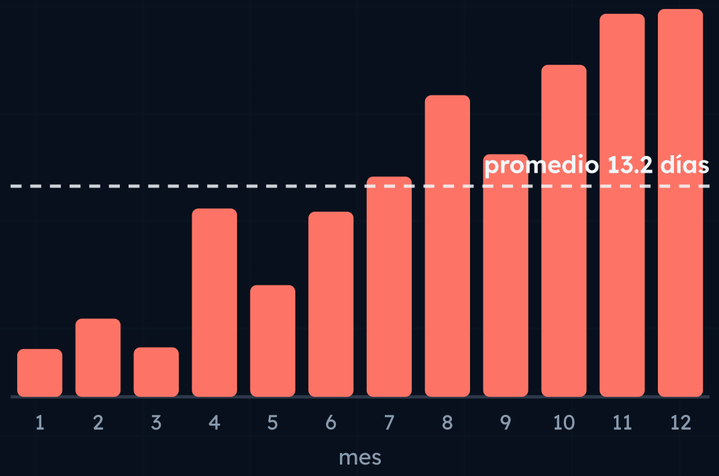
El orden importa

Idea clave 1

Dos socios con **exactamente los mismos 12 números**, en orden contrario. Uno empeora; el otro mejora. Su promedio es idéntico.

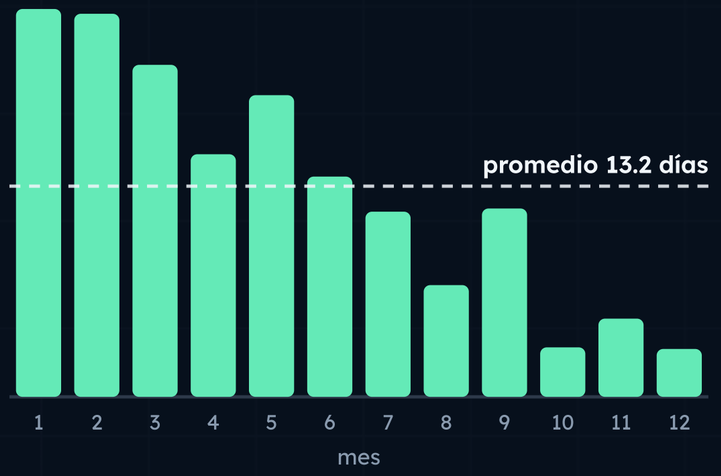
Socio A · empeora

días de atraso de cada mes



Socio B · mejora

los mismos 12 números, al revés



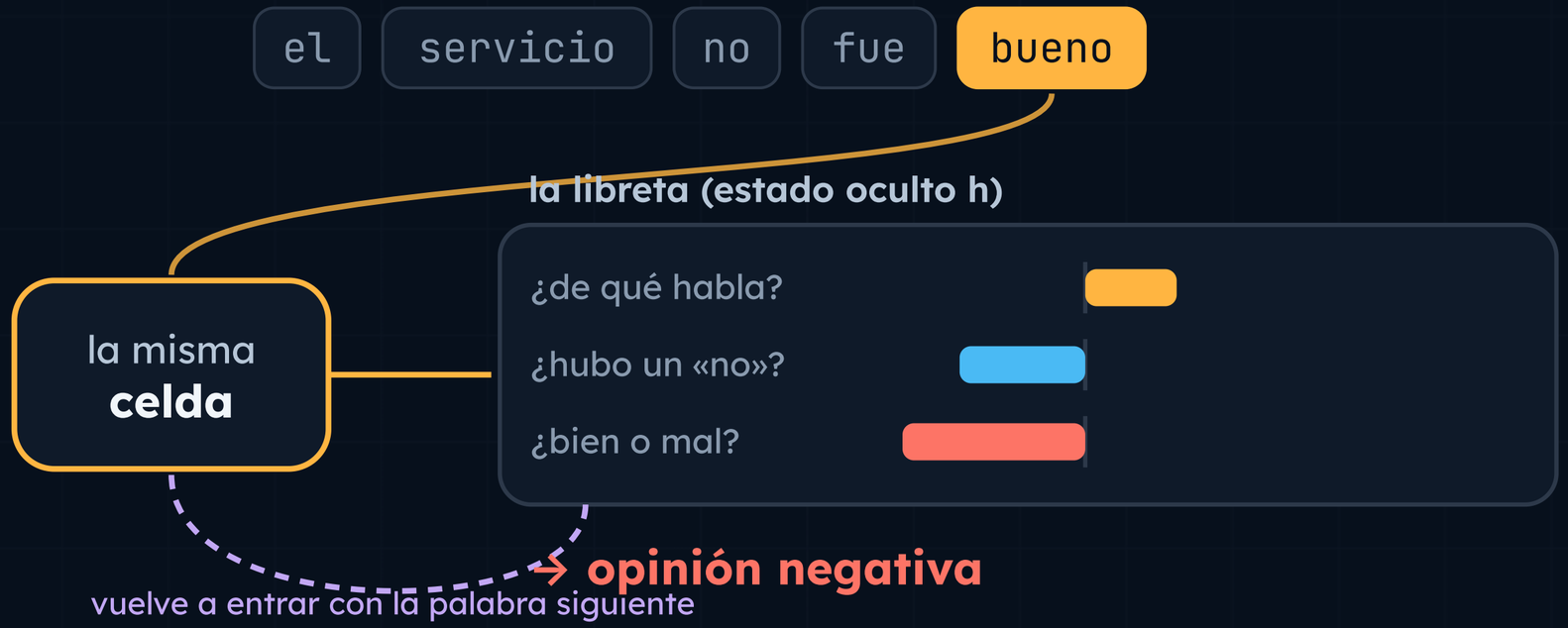
Un modelo que solo mira el **promedio** de los meses (25 parámetros): `48.4 %`. Lo mismo que lanzar una moneda.

Una **RNN** que lee mes por mes (89 parámetros): `100 %`.

Una RNN lee de a uno y anota en su libreta

Idea clave 4

La misma celda recibe la palabra de ahora **y lo que ya tenía anotado**. La libreta (el **estado oculto h**) es pequeña y de tamaño fijo.



Leer la siguiente palabra | Reiniciar | «el servicio no fue bueno» | «el servicio fue muy bueno»

La cuenta de la libreta, con números

Idea clave 4

Es una neurona de la semana 1 con una entrada extra: su propia libreta del paso anterior.

$$h_t = \tanh(w\,x_t + u\,h_{t-1} + b)$$

- h_t: la libreta nueva
- $w\,x_t$: lo que llega ahora
- $u\,h_{t-1}$: lo que ya tenía

Léelo así: mezcla la palabra de ahora con lo anotado y aplasta el resultado entre −1 y 1. **w**, **u** y **b** se aprenden y son **los mismos en todos los pasos**.

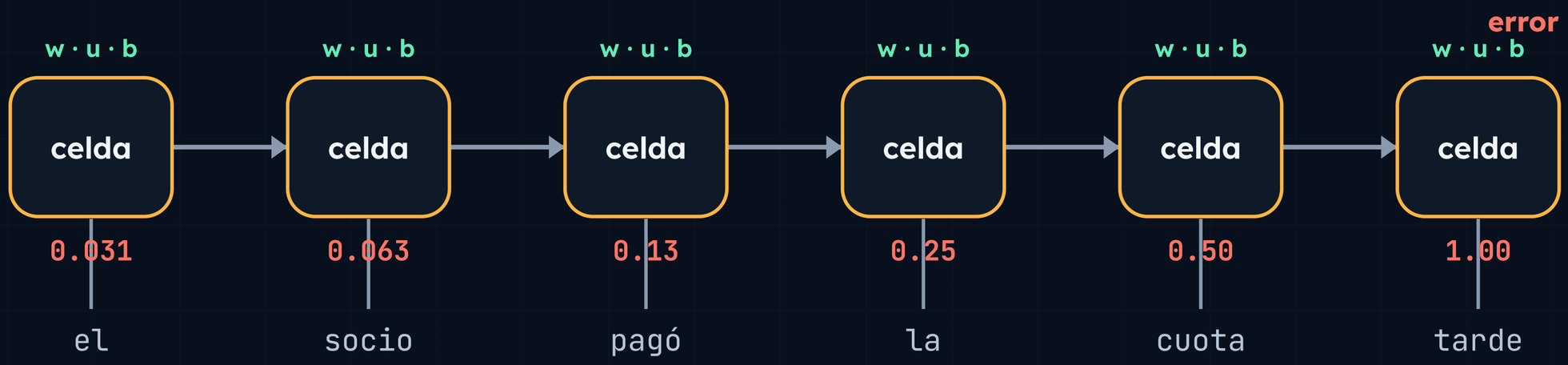
Ejemplo del NB04: **w = 1, u = 0.5, b = 0** y una secuencia que solo trae algo en el primer paso: `x = [1, 0, 0, 0]`.

PASO	X	LA CUENTA	H
1	1	`tanh(1·1 + 0.5·0)`	**0.76**
2	0	`tanh(1·0 + 0.5·0.76)`	**0.36**
3	0	`tanh(0.5·0.36)`	**0.18**
4	0	`tanh(0.5·0.18)`	**0.09**

Cada paso, la huella del primer dato **se reduce a la mitad**. Diez pasos después queda `0.001`.

Desenrollar: la misma celda en cada paso

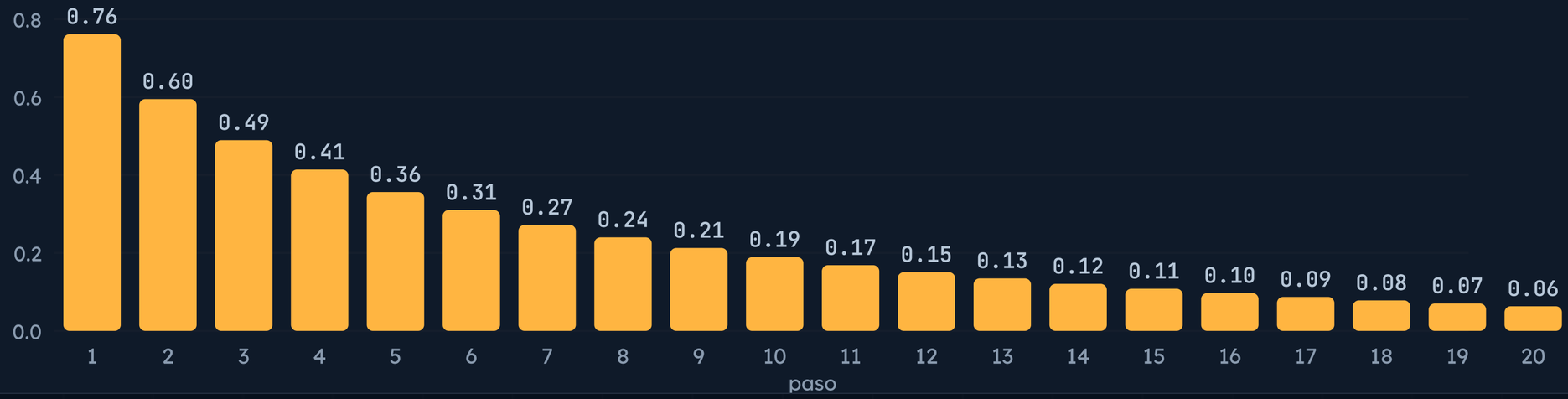
Leer una frase de 6 palabras es como pasar por una red de 6 capas que **comparten los mismos pesos**. Al entrenar, la culpa viaja hacia atrás por todos los pasos.



La libreta se olvida

Idea clave 4

Cuánto queda del primer dato en la libreta, paso a paso. Mueve **u**: cuánto conserva la libreta de un paso al siguiente.



cuánto conserva la libreta: u 0.90

Con u = `0.90`, después de 10 pasos queda `0.19` del primer dato: se apaga más despacio, pero se apaga.

El doble problema: la huella del pasado se apaga al leer, y la culpa se apaga al entrenar. Por eso una RNN simple **no aprende relaciones lejanas**.

LSTM: una libreta con tres compuertas

La LSTM añade una **cinta de memoria** que pasa de un paso al siguiente casi sin cambios, y tres **válvulas** que la red aprende a abrir y cerrar.



1997 Hochreiter y Schmidhuber inventan la LSTM

Si «olvidar» queda casi abierta (≈ 1), el dato viaja muchos pasos **sin apagarse**. Fue la reina de las secuencias durante dos décadas: traducción, voz, series de tiempo.

Las secuencias en Keras

Los mismos 4 pasos de siempre. Lo nuevo: la forma **(pasos, datos por paso)** y una capa que lee en orden.

```
model = keras.Sequential([
    keras.Input(shape=(12, 1)),   # 12 months
    layers.SimpleRNN(8),          # notebook: 8 numbers
    layers.Dense(1, activation="sigmoid"),
])
# with gates: layers.LSTM(32)
```

Input(shape=(12, 1))
12 meses, 1 dato por mes · X.shape → (2000, 12, 1)

SimpleRNN(8)
lee los 12 meses en orden; entrega su libreta final

Dense(1, sigmoid)
con esa libreta decide: ¿empeora?

LSTM(32)
se cambia una línea: cinta y válvulas

Esta red tiene **89** parámetros y acierta el **100 %** del orden de los socios.

Práctica en clase: NB04

En clase · sin nota

40 minutos

En parejas. Todo el código ya viene escrito. Antes de cada celda, apuesta qué va a salir: en la sección 3 casi todos se equivocan.

Qué deberías ver

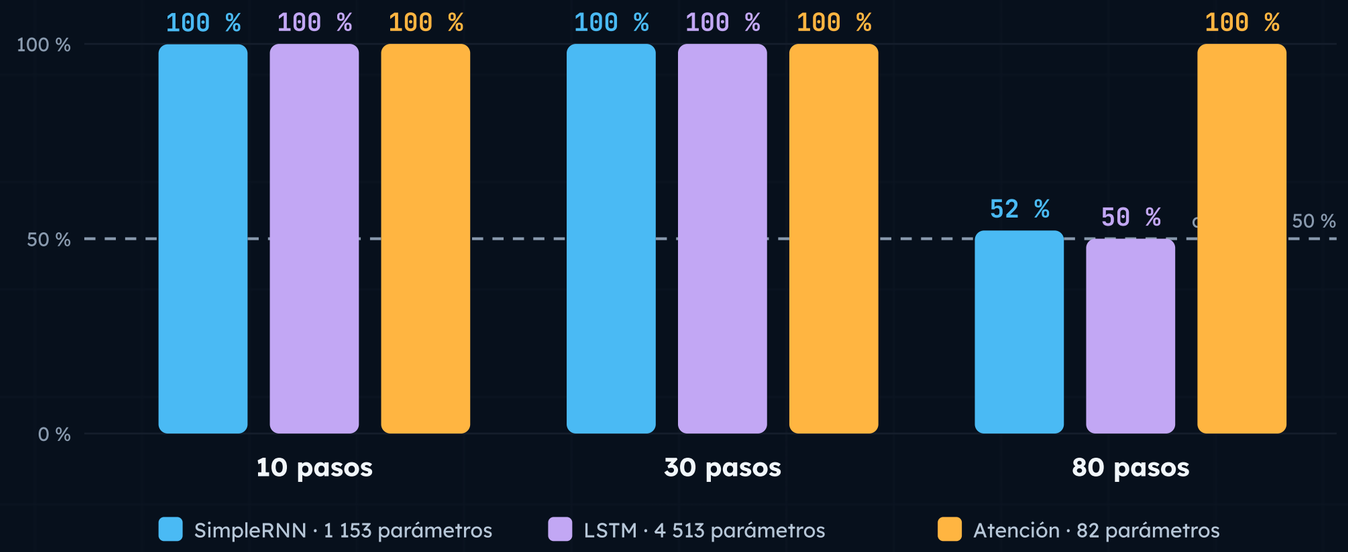
- **1** · La libreta a mano: **0.76** · **0.36** · **0.18** · **0.09**
- **2** · El orden importa: promedio **48.4 %**, RNN **100 %**
- **3** · Examen de memoria con 80 pasos: SimpleRNN **52.1 %**, LSTM **50.0 %**
- **4** · La atención: **100 %** con solo **82** parámetros
- **5** · A mano: «él» mira a «socio» el **79 %**

deeplearningweek1.vercel.app/?nb04

Archivo → **Guardar una copia en Drive**
y ejecuta primero **Configuración**

El examen de memoria: lo que vimos en el NB04

Cada secuencia trae una **marca** en uno de los 10 primeros pasos; la respuesta es el valor marcado. Después vienen muchos pasos de distracción. Mismo presupuesto para todos: 10 épocas.



Con 80 pasos, las recurrentes **adivinan** (52 % y 50 %). La atención acierta el **100 %** con 82 parámetros: no carga el dato en la libreta, **va a buscarlo**.

03

Atención y Transformers

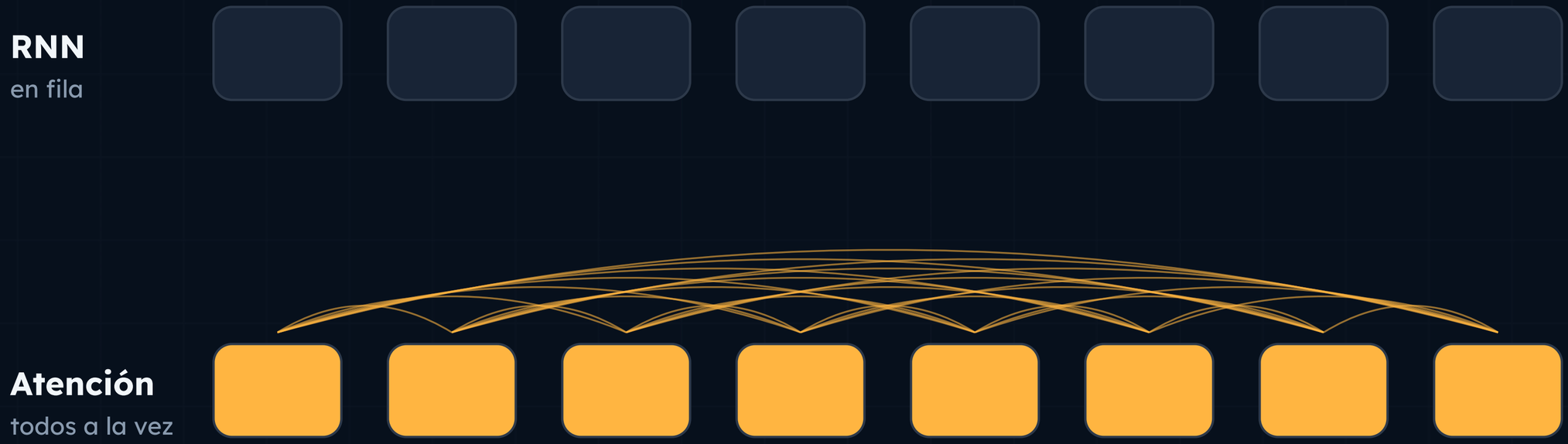
¿Y si en vez de cargar todo en una libreta, cada palabra pudiera mirar directamente a las demás?

- En fila
- Cada palabra mira a todas
- Consulta, clave y valor
- Lo que miró la red
- El precio
- El Transformer
- Los híbridos
- El contrapeso

Sábado · 40 minutos

La RNN lee en fila: no se puede paralelizar

Para leer la palabra 100 necesita haber leído las 99 anteriores. La atención procesa todas a la vez.



Una GPU hace miles de cuentas a la vez: con la RNN espera; con la atención trabaja toda.

La atención: cada palabra mira a todas las demás

Idea clave 5

Y decide cuáles son importantes para entenderla. En la frase de la lectura, «él» mira a «socio», no a «cuota».

«él» mira sobre todo a «socio» (62 %). Haz clic en otra palabra con borde claro.



Cómo decide a quién mirar: consulta, claves y valores

Idea clave 5

Como en una biblioteca: la **consulta** es lo que buscas, las **claves** son las fichas del catálogo y los **valores** son los libros que te llevas.

Vectores de 2 números: **[qué tan persona, qué tan dinero]**. La consulta de «él» es `[1.5, 0]`: busca una persona.

palabra	clave	valor	puntaje	peso
socio	`[2, 0]`	`[1, 0]`	`3.0`	**79 %**
cuota	`[0, 2]`	`[0, 1]`	`0.0`	4 %
él	`[1, 0]`	`[0.5, 0.5]`	`1.5`	17.5 %

El nuevo «él» = 0.79·[1, 0] + 0.04·[0, 1] + 0.175·[0.5, 0.5] ≈ `[0.87, 0.13]`: casi pura persona.

$$\underbrace{\text{puntaje}}_{\text{qué tanto coincide}} = \underbrace{\text{consulta} \cdot \text{clave}}_{\text{el producto punto de la semana 1}}$$

$$\underbrace{\alpha}_{\text{pesos que suman 1}} = \underbrace{\text{softmax}(\text{puntajes})}_{\text{el softmax del viernes}}$$

$$\underbrace{\text{nuevo}}_{\text{lo que entiende ahora}} = \underbrace{\sum \alpha \cdot \text{valor}}_{\text{promedio ponderado}}$$

Lo que miró la red del NB04

La red de atención del examen de memoria, con 80 pasos. Nadie le dijo dónde mirar: aprendió a poner casi todo su peso en el paso marcado.

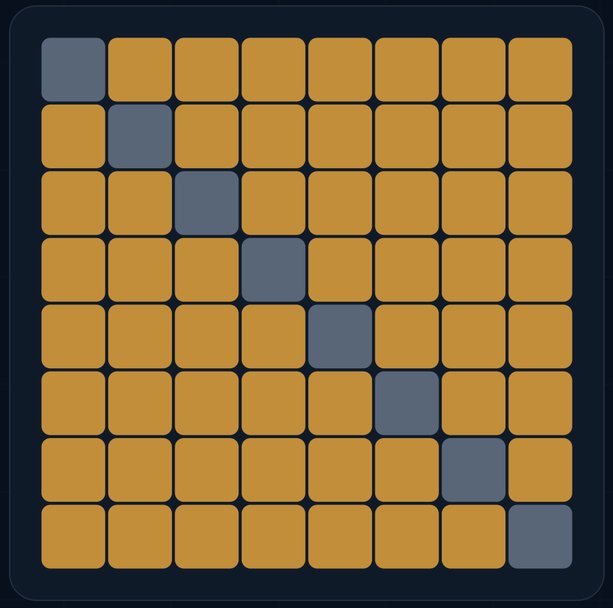


Por eso la distancia no le importa: no carga el dato 80 pasos en una libreta, **lo va a buscar**. Es el «buscador preciso» de la lectura.

El precio: comparar con todas

Idea clave 5

Para generar cada palabra nueva, el Transformer la compara con **todas** las anteriores y guarda una nota de cada una (la **caché KV**). El costo crece con el cuadrado de la longitud.



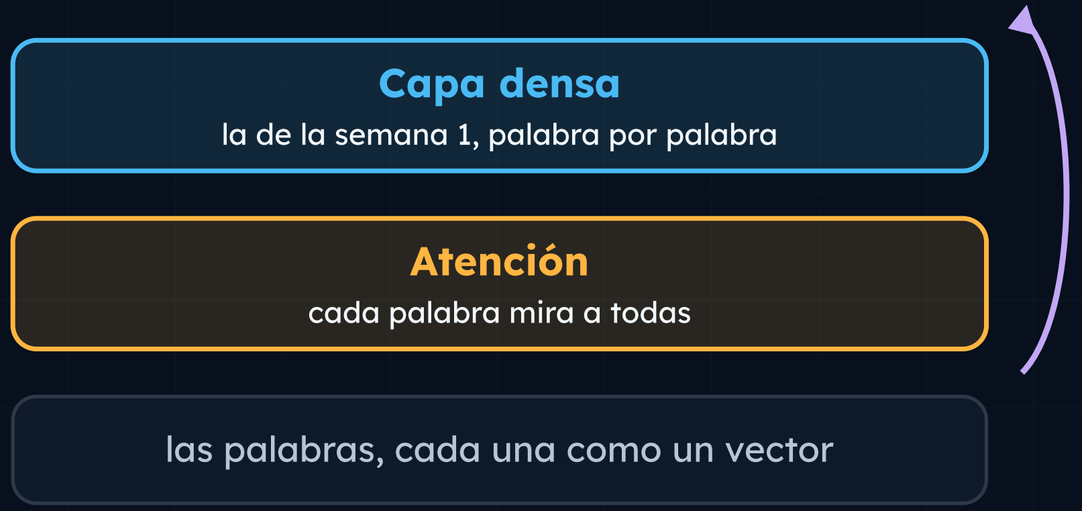
palabras 8

8 palabras → **64** comparaciones. El doble de palabras, **cuatro veces** más cuentas.

PALABRAS	ES COMO	COMPARACIONES
1 000	una carta	1 000 000
10 000	un informe	100 millones
100 000	un expediente	10 000 millones
1 000 000	una biblioteca pequeña	1 billón (10^{12})

El Transformer: atención apilada

En 2017, un equipo de Google propuso abandonar la recurrencia por completo. Su idea central: la atención.



2017 Vaswani et al. · «Attention is all you need»

Ventaja 1

no hay libreta que se llene: puede consultar cualquier parte del texto con precisión

Ventaja 2

todo se calcula en paralelo: se pudo entrenar con cantidades de datos antes impensables

De ahí salieron **BERT**, **GPT** y prácticamente todos los modelos de lenguaje actuales.

Tendencia: las recurrentes vuelven, mezcladas

Lectura de tendencia

Libretas modernas que se entrenan en paralelo (**Mamba**, 2023; **xLSTM 7B**, 2025).
La tendencia más fuerte no fue reemplazar al Transformer, sino **mezclarlo**.

2024

Jamba · AI21 Labs

Una capa de atención por cada siete de Mamba

2025

Nemotron-H · NVIDIA

Cambia ~92 % de las capas de atención por Mamba-2: hasta 3 veces más rápido

2025

Kimi Linear · Moonshot AI

Hasta 75 % menos memoria de caché y 6 veces más rápido con un millón de tokens

2026

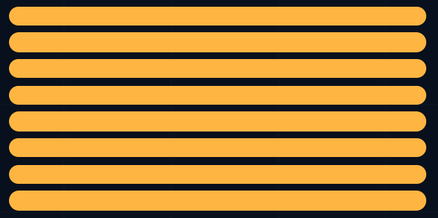
Qwen3.5 · Alibaba

Tres capas de atención «lineal» por cada capa de atención completa

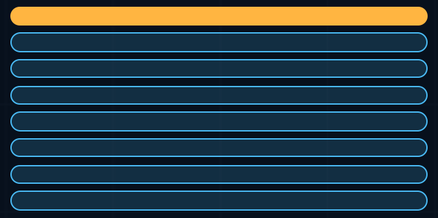
2026

Nemotron 3 Super · NVIDIA

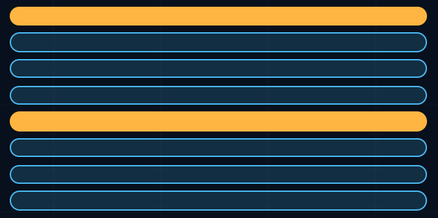
Híbrido Mamba–atención: hasta 2.2 veces más rendimiento



Transformer
todas las capas son de atención



Jamba (2024)
1 de cada 8 capas es de atención



Qwen3.5 (2026)
1 de cada 4 es atención completa

Como un buen estudiante preparando un examen

Apuntes resumidos de casi todo (capas recurrentes, azul) y, de vez en cuando, vuelve al libro por un dato exacto (atención, ámbar).

El contrapeso: lo que la libreta no puede guardar

Lectura de tendencia

Que algo sea más barato no significa que sea igual de bueno.

La libreta tiene un límite

Un Transformer pequeño puede copiar textos muy largos de forma exacta; un modelo con estado de tamaño fijo está limitado por lo que cabe en ese estado. Si te dictan un teléfono largo y solo anotas un resumen, pierdes dígitos.

Jelassi et al., 2024

Los benchmarks pueden engañar

MiniMax volvió a la atención completa en su modelo M2: su híbrido igualaba en las pruebas tradicionales, pero a mayor escala mostró «claros déficits» en razonamiento complejo de varios pasos.

MiniMax, 2025

No hay consenso

A comienzos de 2026 conviven Transformers clásicos, atención comprimida o dispersa y diversos híbridos: «ya nadie se pone de acuerdo sobre la atención».

Labonne, 2026

Debate en el chat

¿Vale la pena ganar de 2 a 6 veces en velocidad si se arriesga la capacidad de recordar con exactitud y razonar en varios pasos? Para un hospital o una cooperativa con pocas GPU, ¿qué elegirían?

04

Elegir la arquitectura

No gana la arquitectura de moda: gana la que hace la suposición correcta sobre tus datos, y se comprueba con una comparación justa.

El mapa · Tres preguntas · Tu proyecto

Sábado · 30 minutos con la actividad

Cada dato con su arquitectura

Cada una es una suposición distinta. Las lecturas del sílabo amplían las tres últimas: Transformers, GNN y GAN.

TABLA

Densa (MLP)

Todo se relaciona con todo.

Socios de una cooperativa, ventas por cliente.

IMAGEN

CNN · o ViT

Lo cercano importa; el patrón puede estar en cualquier parte.

Rayos X, cultivos fotografiados con dron.

SECUENCIA CORTA

RNN · LSTM

El orden importa; basta un resumen.

Pagos de cada mes, lecturas de sensores.

TEXTO LARGO

Transformer · o híbrido

Cada palabra puede necesitar a cualquier otra.

Contratos, expedientes, chatbots.

GRAFO

GNN

Importan las conexiones entre elementos.

Transferencias entre cuentas, moléculas, redes de socios.

CREAR DATOS

GAN · difusión

Aprender a generar ejemplos nuevos que parezcan reales.

Imágenes sintéticas, datos para entrenar.

Y se mezclan: una CNN puede leer cada foto de un video y una RNN el orden de las fotos; los híbridos mezclan libreta y atención.

Tres preguntas antes de elegir

La habilidad más valiosa no es conocer la arquitectura de moda, sino saber qué suposición hace cada una, qué cuesta y cómo comprobar si es la adecuada.

1. **¿Qué estructura tienen mis datos?**
   Tabla, imagen, secuencia, texto largo o grafo. La forma `.shape` ya da una pista.

2. **¿Qué cuesta?**
   Datos para entrenar, parámetros, tiempo de entrenamiento y memoria al usarlo todos los días.

3. **¿Cómo lo compruebo?**
   Contra una línea base simple y con una comparación justa: mismos datos, mismo presupuesto, varias semillas.

Empieza simple. Si una densa ya resuelve tu tabla, no necesitas un Transformer. En el NB04, una sola corrida de la LSTM con 50 pasos ganaba o perdía según la semilla.

Actividad: la arquitectura de tu proyecto

15 minutos en salas · 5 de puesta en común

En salas, con tu grupo de titulación. Al final, cada grupo pega **4 líneas** en el chat, una por pregunta.

1 · Sus datos

¿Qué forma tienen? Escriban su `.shape` : (ejemplos, …).

2 · La suposición

¿Qué arquitectura hace la suposición correcta, y por qué?

3 · La línea base

¿Contra qué modelo simple la van a comparar?

4 · La comparación justa

¿Qué dejarían igual para que la comparación sea justa?

Ejemplo: «(5 000, 12, 3): 12 meses de pagos con 3 datos por mes → LSTM, porque importa la tendencia; línea base: una densa con el promedio; justo: mismos meses de prueba y 3 semillas».

Quiz final: ¿cuánto avanzamos? No es nota

10 preguntas tipo Kahoot. Vuelven las 5 ideas clave del pre-quiz: así vemos tu avance.

1. Abre **deeplearningweek1.vercel.app/?final2**
2. Usa **el mismo nombre** que en el pre-quiz del viernes
3. Pega tu código en el chat de Teams



deeplearningweek1.vercel.app/
?final2

La semana en un mapa

Todo lo de la semana, en el orden en que lo vimos. Borde amarillo: las 5 ideas clave.

VIERNES
Una foto es una tabla de 784 números

IDEA 1
cada arquitectura es una suposición

IDEA 2
la lupa: un filtro de 3 × 3 recorre la foto

IDEA 3
compartir el filtro: 8 veces menos parámetros

SÁBADO
en una secuencia el orden importa

IDEA 4
la libreta resume… y se olvida

LSTM
una cinta con tres válvulas

IDEA 5
la atención mira a todas, a un precio n^2

TENDENCIA
híbridos: apuntes y libro

CONTRAPESO
la libreta no lo guarda todo

ELEGIR
suposición, costo y comparación justa

SEMANA 3
robustez, transfer learning y explicabilidad

Lo que viene: semana 3 Semana 3

Robustez, transfer learning, optimización y explicabilidad.

Robustez

Al mover la prenda 4 píxeles, las dos redes se cayeron. ¿Cómo se entrena una red que aguante? Aumento de datos y regularización.

Transfer learning

Usar una CNN o un Transformer ya entrenado con millones de ejemplos y ajustarlo a tus pocos datos.

Explicabilidad

¿Qué miró la red para decidir? Ya vimos los mapas de los filtros y los pesos de atención: es el comienzo.

Para repasar, y gracias

Guía de estudio	los 4 bloques con ejemplos, figuras y ejercicios resueltos
Slides	estas mismas, interactivas: mueve los deslizadores en casa
NB03 y NB04	CNN y secuencias: todo escrito, para leer y experimentar
Lectura	«¿El regreso de las redes recurrentes?» y su encuesta en Canvas

deeplearningweek1.vercel.app/?guia2

Gracias